

Airframe

Hackathon Challenge - IT / Network Engineering - Wireless Infrastructure



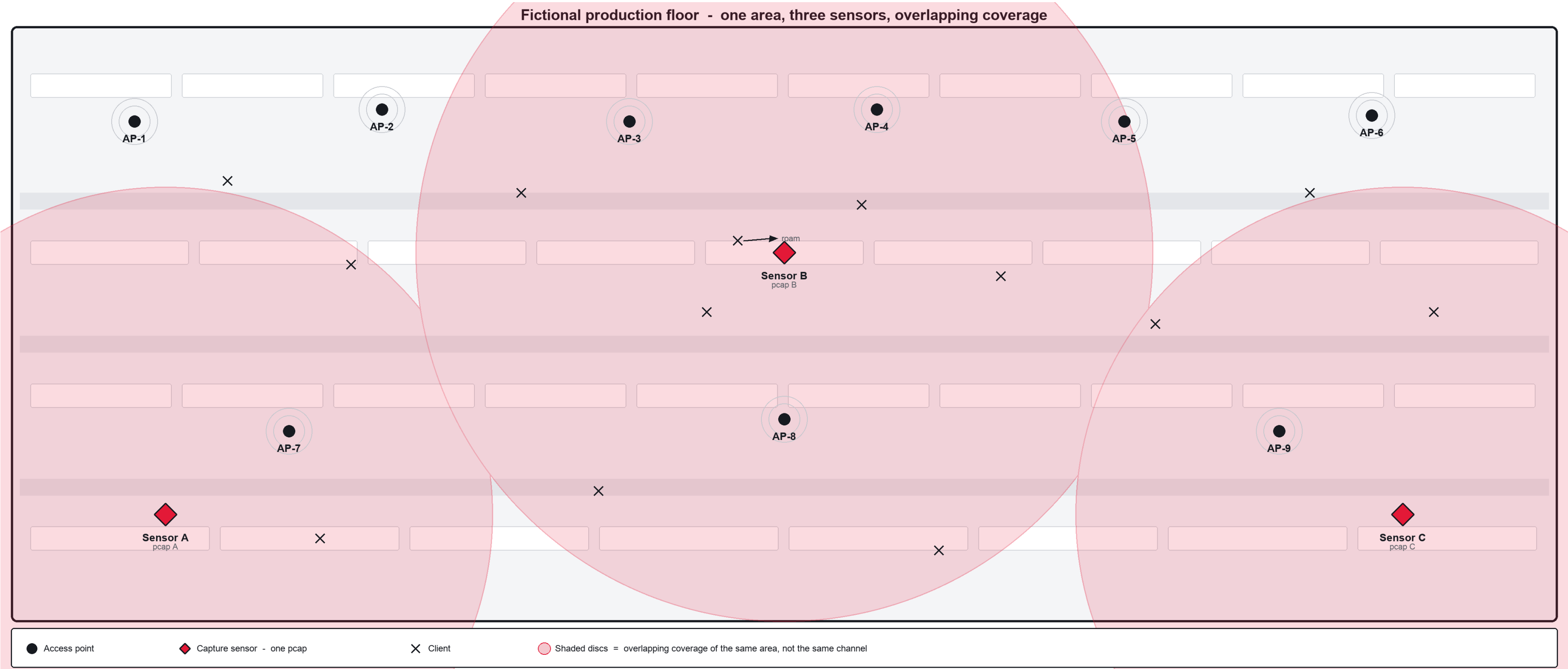
Challenge Airframe

Nearby sensors, overlapping air - find what is going wrong, and show it

Field	Details
Challenge Title	Airframe
Department / Owner	IT / Network Engineering - Wireless Infrastructure
Background & Context	Tesla campuses run dense wireless for production, tools, vehicles, employees, and IoT. A fleet of sensors captures 802.11 continuously. Captures are truncated to protocol headers (802.11 management/control and 802.1X/EAP) - payloads and layer-3 traffic are not visible. Sensors in the same area hear overlapping views of the same air, so the same client or event can appear in more than one capture.
Problem Statement	From a set of related header-only captures, it is hard to see what is actually going wrong on the air: which clients are failing, where the channel is congested, and whether nearby sensors are looking at the same event. Failures hide as breaks in the 802.11 / 802.1X sequence, not as a single obvious frame.
Goals / Objectives	Ingest the provided captures into a pipeline (how is up to the team). Use them to find wireless issues. Cross-compare captures from nearby sensors. Present findings in a dashboard - graphs, events, or similar; the form is up to the team.
Data & Tools Provided	A set of header-only pcaps taken from different sensors in the same area, with some relationship between them. Wireshark/tshark, Python, and libraries on the hackathon image. How the files are fed into the pipeline is up to the team.
Constraints / Out of Scope	See the last slide. In short: header-only analysis, no transmitting on the air, no payload or layer-3 inspection.
Expected Deliverable	A working prototype that reads the captures, surfaces issues, uses more than one sensor view where it helps, and shows findings on a dashboard. A short pitch. Optional extra: a design (not a build) for doing this at the scale of thousands of sensors and access points.

Same floor, many vantage points

We share multiple pcaps of the same area, each from a different sensor



What to look for

Examples - not a checklist. Use the captures, and use more than one sensor when the views overlap.

Example	What it can look like in the captures
Join / auth / association	A client that never completes authentication or association - reject, timeout, or the sequence stops mid-flow.
802.1X / EAP and 4-way handshake	EAP failure or timeout; 4-way handshake that stalls, misses EAPOL M1-M4, or fails MIC. Visible from headers; the data payload after this is not in the capture.
Roam vs deauth loop	A single deauthentication as part of a normal roam is expected. A client bouncing in a deauth/disassoc loop is not. Nearby sensors may each see part of the same roam.
Retries, missing ACKs, congestion	High retry rates or missing ACKs visible from 802.11 headers; many clients contending; overlapping BSS. A sensor only sees the channel it was capturing on - nearby sensors may be on different channels.
Storms and sticky clients	Probe or authentication bursts; a client that stays on a weak AP and keeps failing. Cross-check the same MAC across pcaps from the same floor.
Same event, two sensors	A client roam or a failing join may be split across nearby pcaps. Sensors in the same area do not necessarily share a channel, so use overlap on the same clients and APs where it exists, and treat one client as one finding.

Extra: design at scale

Optional, and design only - do not build this during the hackathon.

Field	Details
What this extra is	If the prototype works on the provided captures, the next step is to design - on paper or in the pitch - a platform that could do the same job with thousands of sensors and access points. Do not implement it.
Ingestion	How captures (or capture streams) would arrive from a large sensor fleet, be identified by sensor, and land in the pipeline without dropping or mixing sites.
Throughput and fan-in	Header-only frames still add up. Think about volume, backpressure, how much you keep vs discard, and how overlapping sensors in one area are correlated without storing everything twice.
Sensor lifecycle	How a sensor is added, identified, and retired; how you know it is alive, silent, or falling behind; how firmware or config would be handled. The laptop demo does not have to run any of this.
What we want to see	A clear architecture sketch and the trade-offs you chose. The stack is yours to propose. There is no prescribed design.

Out of scope and limitations

What not to do. Analysis stays on the captures you are given.

Field	Details
Out of scope	No access to production controllers, access points, or sensors. No transmitting on the air. No changes to campus Wi-Fi. Do not build an access point, a wireless network, or a controller.
What you cannot see	Captures are truncated to 802.11 management/control and 802.1X/EAP. Encrypted data payloads are not there. Layer-3 and above (DHCP, DNS, ARP, application traffic) is out of scope. Do not assume payload visibility.
Limitations	Work from the files provided. Do not remove captures from the approved environment. MAC addresses, EAP identities, and SSIDs are sensitive - mask or hash them in the dashboard and in the pitch.
Scale extra	The scalable platform is a design exercise only. Do not try to run thousands of sensors on a laptop. The on-site demo is the prototype on the provided captures.
Judgment	Expected protocol behavior (for example a normal roam deauthentication) is not a failure. The same client seen by more than one nearby sensor is one event, not three incidents.